HANDS-ON 6 · DAY 2 / AFTERNOON · GROUP'S SHEET

Do the past days tell the next one?

The case. A treasury team buys an index fund once a month, on a day of its choosing. If the Dow Jones's past days told whether the next trading day will be lower, the team could pick a cheaper day. Before anyone builds a forecast, read the series: how much does each day resemble the one before, and does any weekday stand out?

DATA CARD · DOWJONES102425.CSV

What a row is. One trading day of the Dow Jones Industrial Average. 256 rows, 6 columns, no missing value: `Date` (year-month-day), `Open`, `High`, `Low` and `Close`, the index's first, highest, lowest and last price of the day in points, and `Volume`, the shares traded.

The series read. `Close`, the index at the day's end. There is no target: this hands-on reads the series, it trains no model.

How the rows were chosen. Every trading day from 1 October 2024 to 8 October 2025: no weekend, and 11 weekdays absent, the market's holidays (25 December, 1 January, 4 July, …).

Its history. Downloaded for the previous edition of the module and used unchanged. A day's change is its close minus the previous trading day's: after a weekend, Monday minus Friday.

Source and licence. stooq.com, symbol `^DJI`, daily. No licence is recorded: used in class, never republished.

From panel 4. `bike-rentals.csv`: Capital Bikeshare's daily rentals in Washington DC, every day of 2011 and 2012; `Casual` and `Registered` riders add up to `Rentals`. H. Fanaee-T (2013), UCI dataset 275, CC BY 4.0.

file	rows	period	columns	used in
`dowjones102425.csv`	256 trading days	10.2024 to 10.2025	`Date`, `Open`, `High`, `Low`, `Close`, `Volume`	steps 1 to 3
`bike-rentals.csv`	731 days	2011 and 2012	`Date`, `Rentals`, `Casual`, `Registered`	panel 4

STEP 1 · LOAD AND CHART THE INDEX

1 ☐ **Upload the index**
In a new project: **+ Dataset** → **Upload your files**, `dowjones102425.csv`.

Rows: _____ first and last `Date`: _____

2 ☐ **Check that the date is a date**
In the dataset, under each column's name, Dataiku shows the kind of value it found there. Under `Date` it should read **Date only** (it may show **Text** for a moment): the chart's time axis and the weekday in step 2.2 need a date.

3 ☐ **Chart the close**
On the dataset: **Charts**, a **Lines** chart, `Date` on the X axis, `Close` on the Y axis.

Does the close come back to one level, or wander? _____

STEP 2 · PUT EACH DAY BESIDE THE DAYS BEFORE

1 ☐ **Lag the close**
On the uploaded dataset: **+ Recipe** → **Window**. In **Windows definitions**, order the rows by `Date`, ascending; in **Aggregations**, on `Close`, tick **Lag** with offsets `1,2`; **Run**.

Names of the two new columns: _____

2 ☐ **Compute the changes**
On the lagged dataset: **+ Recipe** → **Prepare**, a step **Formula** `change` = `Close` minus the lag-1 column, a step **Formula** `previous_change` = the lag-1 column minus the lag-2 column, then **Extract date elements** on `Date`, the day of week; **Run**. Then **Charts**: a **Lines** chart of `change` by `Date`.

3 ☐ **Correlate**
On the dataset of changes: **Statistics** → **+ Card** → **Correlation matrix** on `Close`, the lag-1 column, `change` and `previous_change`; in the card's settings, **Pearson** (Spearman is the default).

Close with the day before: _____ change with the change before: _____

STEP 3 · THE WEEKDAY CHECK

1 ☐ **Average the change by weekday**
On the dataset of changes: **+ Recipe** → **Group**, grouped by the day of week; on `change`: **Avg**, **Std. dev.** and **Count**; **Run**. In its output, `day_of_week` 1 is Monday; the days are `change_count` (`count` also counts the first day, which has no change).

`change`	**Mon**	**Tue**	**Wed**	**Thu**	**Fri**
mean					
std. dev.					
days					

2 ☐ **Compare each mean with its spread**
On the Group recipe's output dataset.

Does any weekday's mean stand out of its standard deviation? _____

3 ☐ **Answer the treasury team**
From the table and the correlations of step 2.3.

What do you tell the treasury team: do the past days point to a cheaper day to buy, and why?

4 · RUN THE SAME CHECK ON THE BIKE RENTALS

1 ☐ **Upload the rentals and read their dates**
+ Dataset → **Upload your files**, `bike-rentals.csv` (its `Date` reads **Date only**, as in step 1.2); then **+ Recipe** → **Prepare**: **Extract date elements** on `Date`, the day of week and the month; **Run**.

2 ☐ **Average by weekday, then by month**
+ Recipe → **Group** grouped by the day of week (1 is Monday), on `Rentals`: **Avg** and **Std. dev.**; then a second Group recipe on the same dataset, grouped by the month.

`Rentals`	**lowest mean**	**highest mean**	**a typical std. dev.**
by weekday			
by month			

Does a weekday or a season stand out of its spread here? What would such a pattern have looked like in the index's weekday table of step 3?